

# Deciding FRACTRAN holdouts with rigid phase certificates: what makes a small machine tractable, and where the line falls

(authorship placeholder — see §9)

August 2026

## Abstract

Small, fully explicit machines can encode open problems in number theory. This paper is about the boundary between those that do and those that merely look as though they do. We work in FRACTRAN, Conway’s minimal universal language, on the live BBf(23) Busy Beaver holdout list. We isolate the property that separates tractable machines from the rest — we call it *rigidity* — turn it into a certificate class with a decidable checking problem, prove the checker sound, formalize that soundness proof in Lean 4, and then *search* for certificates automatically. The result is an end-to-end decision procedure that decides 25 of the 694 machines on the refined holdout list, sixteen of them not previously decided by anyone. Nine of the 25 additionally carry individual, **sorry**-free Lean proofs.

We have tried to write the paper we would have wanted before starting. The mathematics involved is elementary; what is not obvious is *which* elementary facts matter, and why the method stops exactly where it does. Three lessons generalize beyond FRACTRAN. First, priority is what makes these machines universal, and closed-form orbits are what make individual ones tractable — rigidity is the decidable margin of a holdout list, not an accident of which machines were studied. Second, guard checking over a parametric family reduces to two facts of independent interest: an extreme-point principle for jointly affine functions, and the decidability of eventual positivity in a small expression class that sits just inside the frontier of the Positivity problem. Third — and this one cost us a wrong proof — verifying that a competing rule is *disabled* across a run cannot be done by testing its guard’s negation at the endpoints; one conjunct must be pinned. We give a machine-checked counterexample.

We also report a negative result that saves future effort: the holdout list is already canonical under the only prime-relabelling symmetry that transfers the halting question, so no machines can be removed by relabelling alone.

## Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>2</b> |
| <b>2</b> | <b>FRACTRAN is a priority counter machine</b>                    | <b>3</b> |
| <b>3</b> | <b>Rigidity: the decidable margin</b>                            | <b>3</b> |
| <b>4</b> | <b>Certificates: the proof object</b>                            | <b>4</b> |
| <b>5</b> | <b>Three principles that make checking finite</b>                | <b>4</b> |
| 5.1      | The extreme-point principle . . . . .                            | 5        |
| 5.2      | Eventual positivity, and a frontier we sit just inside . . . . . | 5        |
| 5.3      | Disabling a rival: pin one conjunct . . . . .                    | 5        |
| <b>6</b> | <b>Finding certificates</b>                                      | <b>6</b> |

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>7</b> | <b>Where the method stops</b>                      | <b>6</b> |
| 7.1      | Thin margins: one step past the boundary . . . . . | 6        |
| 7.2      | Symmetry is exhausted: a negative result . . . . . | 6        |
| 7.3      | The wall . . . . .                                 | 7        |
| <b>8</b> | <b>The Lean development</b>                        | <b>7</b> |
| <b>9</b> | <b>Reproducibility and provenance</b>              | <b>7</b> |

# 1 Introduction

A FRACTRAN program is a finite ordered list of positive rationals. The state is a positive integer  $n$ ; a step replaces  $n$  by  $n \cdot p/q$  for the *first* listed fraction  $p/q$  with  $q \mid n$ ; the program halts when none applies. That is the entire language, and it is Turing-complete [2].

The Busy Beaver variant asks, among all programs of description size at most  $k$ , for the longest finite run. As with Turing machines, the difficulty is not finding long runners but *certifying* the non-halters: a single program that runs forever must be proved to do so. BBf(23) is the current frontier, with 21,233 machines officially undecided as of 10 July 2026 and a refined list of 694 [14].

The reason a specific tiny program can resist proof is worth stating plainly, because it is the subject of this paper. Some machines have orbits with closed forms; you can write down the state after the  $i$ -th phase as a formula in  $i$ . Others choose their next move from arithmetic detail — a digit, a residue, a valuation — that changes unpredictably as the state grows. For the first kind, non-halting is a finite algebraic identity. For the second, it is a Collatz-type problem, and Collatz-type problems in this generality are undecidable [1, 8].

**What a reader should take away.** We have organized the paper around ideas rather than around our machines.

- §2: FRACTRAN *is* a priority counter machine, and reading it that way immediately explains where its power comes from — and locates it against a well-studied decidability boundary.
- §3: what rigidity is, why it is the right dividing line, and why it is a property of the *dynamics* rather than of our cleverness.
- §4: certificates as the natural proof object, and the precise sense in which checking one is decidable.
- §5: the three elementary principles that make the checking finite — including the one we first got wrong.
- §6: how to *find* certificates, which is a different and much less rigorous activity than checking them, and why that asymmetry is the right architecture.
- §7: where the method stops, what a machine looks like just past the boundary, and a measured negative result about symmetry.

**Epistemic labels.** Every claim is marked [proved] (hand proof), [proved, Lean-verified] (machine-checked in Lean 4), [checker-verified] (verified by our checker, whose soundness is proved and Lean-verified, but not individually formalized), or [measured] (a measurement over a stated finite domain). The distinction between [proved, Lean-verified] and [checker-verified] is real and we keep it visible.

## 2 FRACTRAN is a priority counter machine

Factor the state. Writing  $n = \prod_j p_j^{v_j}$ , the divisibility test  $q \mid n$  becomes a conjunction of lower bounds on the exponents  $v_j$ , and multiplying by  $p/q$  becomes adding a fixed integer vector. So a FRACTRAN program is exactly a finite set of guarded vector-addition rules over  $\mathbb{N}^d$ , with the guards being lower bounds and the rules tried in a fixed priority order.

This is not a new observation — it is how Conway’s universality proof works, and the machine model is Minsky’s [3] — but it is worth dwelling on, because it places FRACTRAN precisely against a decidability boundary that is one of the most studied in computer science.

Drop the priority. What remains — guarded vector addition where any enabled rule may fire — is a *vector addition system*, equivalently a Petri net [4]. For those, reachability is **decidable**, a celebrated result [5, 6] whose exact complexity was settled only recently as Ackermann-complete [7]. Add priority — “fire the first enabled rule” — and the model becomes Turing-complete, so reachability is undecidable. FRACTRAN’s entire computational power sits in the word *first*.

That reframing pays off immediately. It tells us that any hope of a general decision procedure is misplaced, so the question becomes which *fragment* is decidable; and it tells us where to look, because priority only matters when two rules are simultaneously enabled. A machine whose orbit keeps the competing guards separated is, in effect, running a plain vector addition system, and those we can reason about. The certificates of §4 are, from this angle, exactly witnesses that the priority never has to be adjudicated in a way that depends on unbounded arithmetic detail.

## 3 Rigidity: the decidable margin

Consider the first steps of a typical holdout. A rule fires while some coordinate lasts, then another takes over; the pattern repeats at a larger scale; and the question is whether the pattern always repeats. Two things can happen.

The pattern can be *rigid*: the sequence of rules in each repetition is the same word, with the repetition counts given by a formula in the phase index. Then the state at the start of each phase is a formula too, and non-halting becomes: do these formulas keep the guards positive forever? That is an algebraic question about finitely many expressions.

Or the pattern can *consume digits*: the rule chosen at some point depends on the residue of the current state modulo something that grows, or on the 2-adic valuation of a quantity that changes at every phase. Then no formula in the phase index describes the word, and the orbit is the branching, pseudorandom object familiar from  $3x + 1$  [9]. For such maps, the “expected” behaviour is easy to compute and hard to prove — the state of the art on Collatz itself is an almost-all theorem [10], and even that says nothing about any *particular* orbit.

The distinction is not about our ingenuity. It is a property of the machine, it is visible in a bounded simulation (§6), and it predicts which machines have already fallen to earlier work. Our census of the BBf(23) list bears this out: rigid machines are a sizeable minority of the official list and a *tiny* minority of what survives after the community’s deciders have run — exactly what one expects if rigidity is the decidable margin being steadily consumed.

| list                      | rigid | polynomial | digit-consuming |
|---------------------------|-------|------------|-----------------|
| official, 21,233 machines | 3,253 | 6,797      | 11,183          |
| refined, 694 machines     | 9     | 10         | 675             |

Table 1: Structural census of the holdouts, by a bounded-simulation detector. [\[measured\]](#)

## 4 Certificates: the proof object

If a machine is rigid, what exactly should we write down?

The instinct from program verification is to look for an *inductive invariant*: a property preserved by every step, implying non-halting. That is not quite the right shape here, because the interesting property is not preserved step by step — it is restored once per phase, after thousands of steps that violate it. The right object is therefore a *boundary family* together with a proof that it reproduces itself.

**Definition 4.1** (rigid phase certificate). Let  $\text{EXP} = \{a + bn + c2^n : a, b, c \in \mathbb{Q}\}$ . A certificate consists of: an *entry*  $(E, i_0)$  — the machine reaches the  $i_0$ -th boundary in  $E$  steps from its start; a *boundary family*  $B(n)$  with each coordinate in  $\text{EXP}$ ; and, for each residue class of  $n$  modulo 2, a list of *stages* describing the phase word: a *run* of one rule repeated an  $\text{EXP}$  number of times, or a *block* — a fixed short word repeated an  $\text{EXP}$  number of times, whose start state is affine in the block index.

Why  $\text{EXP}$ ? Because it is the smallest class closed under what the phases actually do. A run consumes a coordinate that was built up by doubling, so counts like  $2^n - 1$  appear; step counts accumulate linear corrections, so  $n$  appears; and the class must be closed under  $n \mapsto n + 1$  and under integer combinations, which it is. It is also small enough that  $\{1, n, 2^n\}$  are linearly independent over  $\mathbb{Q}$  — so two states are equal as formulas exactly when their nine coefficients agree, and *symbolic state equality is a finite check*. That single fact is what makes the whole scheme work.

**Theorem 4.2** (soundness). *If the checker accepts a certificate for  $M$ , then  $M$  never halts from its start.* [proved, Lean-verified]

The proof splits into a dynamical core and a geometric part. The core is short enough to state in full.

**Theorem 4.3** (phase induction). *Let  $B : \mathbb{N} \rightarrow \mathbb{N}^d$ . If the machine reaches  $B(i_0)$  from its start, and for every  $i \geq i_0$  there is  $m \geq 1$  with  $B(i) \xrightarrow{m} B(i + 1)$ , then it never halts.* [proved, Lean-verified]

*Proof.* By induction, for every  $k$  the machine completes at least  $k$  steps: the entry is the base case, and each phase contributes at least one more step. A machine that completes arbitrarily many steps never halts.  $\square$

Everything else in the checker exists to establish the hypothesis of Theorem 4.3 for all  $i$  at once.

**Relation to Busy Beaver deciders.** The community’s deciders for Turing machines are broadly of two kinds: those that find a regular invariant of the tape language (closed tape languages, finite-automata reductions), and those that certify a repeating macro-structure (translated cyclers, bouncers). Certificates here are of the second kind, lifted to arithmetic: the “macro-structure” is a phase, and the repetition is witnessed by algebra rather than by tape matching. The precedent for taking such proofs seriously at scale is Coq-BB5 [13], which settled BB(5) with a verified decider plus per-machine certificates — the architecture we follow.

## 5 Three principles that make checking finite

A certificate asserts something about infinitely many phase indices and, within each phase, about a number of steps that grows exponentially. The checker must reduce that to finitely much arithmetic. Three facts do it.

## 5.1 The extreme-point principle

Inside a block the state is affine in the block index  $k$ ; within a run it is affine in the run counter  $t$ . So every guard value has the shape  $a + bk + ct$  over a rectangle  $[0, K] \times [0, L]$ .

**Theorem 5.1** (corner principle). *If  $a + bk + ct \geq 0$  at the four corners of the rectangle, it is  $\geq 0$  throughout; likewise for  $\leq \theta$ . [proved, Lean-verified]*

*Proof.* A linear function on an interval attains its extremes at an endpoint. Apply this in  $k$  along the two extreme rows, then in  $t$  for fixed  $k$ .  $\square$

This is the polyhedral fact that linear programs attain optima at vertices, in its smallest useful instance. It converts “check every one of  $2^n$  steps” into “check four states”.

## 5.2 Eventual positivity, and a frontier we sit just inside

The corners are themselves formulas in EXP, so the checker must decide statements like “ $g(n) \geq 1$  for every  $n \geq n_0$  in an arithmetic progression” for  $g \in \text{EXP}$ .

**Proposition 5.2.** *This is decidable. [proved]*

*Proof.* Write  $g = a + bn + c2^n$ . If  $c < 0$ , or  $c = 0$  and  $b < 0$ , then  $g$  eventually falls below any bound: answer no. Otherwise the increment of  $g$  along the progression is eventually nonnegative, at an explicitly computable index  $N$ ; check  $n_0 \leq n \leq N$  directly.  $\square$

It is worth knowing how narrow the ledge is. EXP consists of linear recurrence sequences of order at most three with the very tame characteristic roots 1, 1, 2. Asking whether a general linear recurrence stays nonnegative is the *Positivity problem*, and it is open in general; decidability is known only at low order, and even there the proofs are delicate and connected to hard Diophantine questions [11]. We are not doing anything clever here — we are working in a class chosen small enough that positivity is trivial. That is a design decision, and it is the honest reason the checker terminates.

## 5.3 Disabling a rival: pin one conjunct

The third principle is the one we got wrong first, and it is the most transferable.

To justify that a certified rule really fires, one must show that every higher-priority rule is *disabled* throughout the run. A rule is disabled when *some* conjunct of its guard fails. The tempting check is: verify “some conjunct fails” at the two ends of the run, and appeal to the corner principle. This is invalid.

**Proposition 5.3.** *There are affine conjunct values  $p + ut$ ,  $q + vt$  and a length  $n$  such that at  $t = 0$  and at  $t = n$  some conjunct fails, yet at an intermediate  $t$  both hold — so the rival rule fires in the middle of a run the check accepted. [proved, Lean-verified]*

*Proof.* Take  $p + ut = t$  and  $q + vt = 2 - t$  with  $n = 2$ : at  $t = 0$  the first vanishes, at  $t = 2$  the second does, and at  $t = 1$  both equal 1.  $\square$

The error is elementary once seen: the corner principle applies to a single affine function, and “disabled” is a *disjunction* of affine conditions, whose truth set is not convex. The fix is to require one fixed conjunct to fail at all corners, which is again a single affine function and so is covered by Theorem 5.1.

We flag this because the same trap is available to any verifier that reasons about guarded transitions by interval or corner testing, and because it has an audible signature: the failing certificate is not detected by more simulation — the machine behaves exactly as expected at both ends of every run — but only by asking which conjunct is doing the work. In the Lean development the discipline shows up as a rule typically needing *two* firing lemmas, because different coordinates disable the same rival in different stages.

## 6 Finding certificates

Checking a certificate is a proof; finding one is a search, and the two should not be confused. We keep them in separate programs, with the checker as the only authority. The miner may fail, and it may propose nonsense; nothing it proposes counts until the checker accepts it, and the checker’s verdict is symbolic in the phase index rather than a range check. Consequently the miner needs no correctness argument at all — a liberating architecture, and the same one used by SAT-based verification and by Coq-BB5.

The search itself is short to describe. Simulate. For each rule, take the positions that begin a maximal run of it; these are boundary candidates, because a rigid machine’s phases begin the same way every time. Fit each coordinate of the candidate boundary states to EXP exactly over  $\mathbb{Q}$  — three unknowns, solved on three phases and required to reproduce all the others. Segment each phase word into runs and blocks by looking for maximal repetitions of a short sub-word. Require the resulting *stage signature* to be identical across phases of the same parity; this is precisely the rigidity test, and it is where digit-consuming machines fail. Fit the repetition counts. Finally — an economy worth noting — do *not* fit the block start states: obtain them by symbolically composing the earlier stages, exactly as the checker will.

|                      | machines | decided   | of these, new |
|----------------------|----------|-----------|---------------|
| refined BBf(23) list | 694      | <b>25</b> | <b>16</b>     |

Table 2: End-to-end sweep: fraction list in, certificate out, checker verdict. No anomalies: every accepted machine was re-checked and independently simulated for 200,000 steps without halting. A sweep of the full official list was running at the time of writing. [\[checker-verified\]](#)

The nine machines we had previously proved non-halting by hand are all rediscovered by the miner from the fraction lists alone, several with shorter entries than we had found — one of them entering at step zero, its start state already being a boundary. The other sixteen are new.

## 7 Where the method stops

### 7.1 Thin margins: one step past the boundary

Every machine we decide survives by a comfortable margin: a coordinate that grows linearly in the phase index, or an invariant preserved exactly. Contrast the one previously decided BB(6) machine of comparable shape, whose halting reduces to the 2-adic valuation of  $2 \cdot 3^i + i + 5$  staying above a linear bound [15]. That orbit is rigid — there is a closed form — but its margin is *thin*: it is a near-coincidence between powers of 2 and powers of 3, and controlling it needs Baker’s theorem on linear forms in logarithms [12]. The published proof accordingly invokes Baker–Wüstholz as an axiom.

So rigidity is necessary but not sufficient: what also matters is whether the surviving quantity has room. Our nine, and the sixteen new ones, have room; that is why nothing beyond guard arithmetic and induction appears anywhere in this paper. It is a useful triage question for any new machine: is the orbit closed-form, and if so, is the margin thick or thin?

### 7.2 Symmetry is exhausted: a negative result

A natural hope for shrinking a holdout list is symmetry: if two programs differ only by renaming primes, deciding one decides the other. There is a catch that turns out to be decisive. Every program starts at  $n = 2$ , so a relabelling  $\pi$  transports the *start* to  $e_{\pi(2)}$ ; the halting-from-2 question transfers only when  $\pi$  fixes the prime 2.



**Proposition 7.1.** *Under the relabellings that fix the prime 2, the official 21,233 machine list has 21,233 distinct classes: **no two machines on the list are related by a sound relabelling**. Under all prime relabellings — which do not transfer the start — the list collapses to 15,600 classes, so 5,633 machines (26.5%) are dynamically conjugate to another entry while remaining genuinely distinct questions. [\[measured\]](#)*

The list is thus already canonical with respect to the only symmetry that helps, and the remaining 26.5% is a mirage: those pairs relate “machine *A* from 2” to “machine *B* from some other prime”. The practical residue is smaller but real — conjugate machines have template-isomorphic certificates, so the expensive part, the phase analysis, transfers even though the decision does not. We record the negative result because it is the kind of thing worth knowing before spending a week on it.

### 7.3 The wall

What is left is digit-consuming, and it is most of everything: 675 of the 694 refined holdouts. For those, no boundary family in EXP — indeed none with a bounded description — tracks the orbit, so the certificate class is empty for them by construction rather than by budget. This is the same wall that stands in front of Collatz. We make no claim to address it, and we would be suspicious of a method that claimed to without saying which of the two behaviours above it had overcome.

## 8 The Lean development

The formalization uses Lean 4 with **no** `mathlib`: core arithmetic, `omega`, and kernel `decide`. It contains no `sorry`, no `admit`, no `native_decide` and no `axiom` declarations; every non-halting theorem depends on exactly Lean’s three core axioms, and the entry lemmas on none.

Two parts are worth distinguishing. `Decider.lean` formalizes the *soundness argument* — Theorems 4.3 and 5.1 and Proposition 5.3 — with four machines re-derived through the abstract theorem to confirm the contract is what the concrete proofs establish. The per-machine files formalize nine machines individually. We do *not* claim the Python checker is verified: what is proved is that no certificate meeting the contract can exist for a halting machine. The sixteen new decisions are [\[checker-verified\]](#), not [\[proved, Lean-verified\]](#); transcribing them is mechanical and is the natural next step, most efficiently done by formalizing the checker itself so that each machine reduces to evaluating a decidable predicate on certificate data.

## 9 Reproducibility and provenance

Code, certificates, logs and the Lean development accompany the paper. The checker runs the certificates in about a second; a falsifier that perturbs boundaries, counts, rule indices and block coefficients produced 540 corrupted certificates, all rejected. Holdout lists are those published by the BBFractran project [\[14\]](#); machine reductions quoted from the bbchallenge wiki [\[?\]](#) appear verbatim beside the code that uses them.

This work was carried out with substantial AI assistance.

## References

- [1] J. H. Conway. Unpredictable iterations. In *Proceedings of the 1972 Number Theory Conference*, University of Colorado, Boulder, 1972, 49–52.
- [2] J. H. Conway. FRACTRAN: a simple universal programming language for arithmetic. In *Open Problems in Communication and Computation*, Springer, 1987, 4–26.

- [3] M. L. Minsky. *Computation: Finite and Infinite Machines*. Prentice-Hall, 1967.
- [4] R. M. Karp and R. E. Miller. Parallel program schemata. *Journal of Computer and System Sciences* 3(2), 1969, 147–195.
- [5] E. W. Mayr. An algorithm for the general Petri net reachability problem. *SIAM Journal on Computing* 13(3), 1984, 441–460.
- [6] S. R. Kosaraju. Decidability of reachability in vector addition systems. In *Proceedings of STOC*, 1982, 267–281.
- [7] W. Czerwiński and Ł. Orlikowski. Reachability in vector addition systems is Ackermann-complete; and J. Leroux, The reachability problem for Petri nets is not primitive recursive. Both in *Proceedings of FOCS*, 2021.
- [8] S. A. Kurtz and J. Simon. The undecidability of the generalized Collatz problem. In *Theory and Applications of Models of Computation (TAMC)*, LNCS 4484, Springer, 2007, 542–553.
- [9] J. C. Lagarias. The  $3x+1$  problem and its generalizations. *American Mathematical Monthly* 92(1), 1985, 3–23.
- [10] T. Tao. Almost all orbits of the Collatz map attain almost bounded values. *Forum of Mathematics, Pi* 10, 2022, e12.
- [11] J. Ouaknine and J. Worrell. Positivity problems for low-order linear recurrence sequences. In *Proceedings of SODA*, 2014, 366–379.
- [12] A. Baker and G. Wüstholz. Logarithmic forms and group varieties. *Journal für die reine und angewandte Mathematik* 442, 1993, 19–62.
- [13] The bbchallenge Collaboration. Coq-BB5: a formally verified determination of BB(5). <https://github.com/ccz181078/Coq-BB5>
- [14] The BBFractran project. Holdout lists and enumeration conventions. <https://github.com/int-y1/BBFractran>
- [15] bbchallenge wiki, machine 1RB1LA\_1RC1RE\_1LDORB\_1LA0LC\_ORFORD\_ORB---. <https://wiki.bbchallenge.org>
- [16] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Proceedings of CADE-28*, LNCS 12699, Springer, 2021, 625–635.